Camelot[9], PDFPlumber[10], and Adobe Acrobat® Pro[11]) and the WYGIWYS model[12] [34]. We crop the test tables from the original PDF for Tabula, Traprange, Camelot, and PDFPlumber, as they only support text-based PDF as input. Adobe Acrobat® Pro is tested with both PDF tables and high-resolution table images (300 PPI). The outputs of the off-the-shelf tools are parsed into the same tree structure as the HTML tables to compute the TEDS score.

A. Implementation details

To avoid exceeding GPU RAM, the EDD model is trained on a subset (399k samples) of PubTabNet training set, which satisfies

$$\begin{aligned} \text{width and height} &\leq 512 \text{ pixels} \\ \text{structural tokens} &\leq 300 \text{ tokens} \\ \text{longest cell} &\leq 100 \text{ tokens.} \end{aligned} \tag{3}$$

Note that samples in the validation and test sets are not constrained by these criteria. The vocabulary size of the structural tokens and the cell tokens of the training data is 32 and 281, respectively. Training images are rescaled to 448×448 pixels to facilitate batching and each channel is normalized by z-score.

We use the ResNet-18 [39] network as the encoder. The default ResNet-18 model downsamples the image resolution by 32. We modify the last CNN layer of ResNet-18 to study if a higher-resolution feature map improves table recognition performance. A total of five different settings are tested in this paper:

- EDD-S2: the default ResNet-18
- EDD-S1: stride of the last CNN layer set to 1
- EDD-S2S2: two independent last CNN layers for structure (stride=2) and cell (stride=2) decoder
- EDD-S2S1: two independent last CNN layers for structure (stride=2) and cell (stride=1) decoder
- EDD-S1S1: two independent last CNN layers for structure (stride=1) and cell (stride=1) decoder

We evaluate the performances of these five settings on the validation set and find that a higher-resolution feature map and independent CNN layers improve performance. As a result, the EDD-S1S1 setting provides the best validation performance, and is therefore chosen to compare with baselines on the test set.

The structure decoder and the cell decoder are single-layer long short-term memory (LSTM) networks, of which the hidden state size is 256 and 512, respectively. Both of the decoders weight the feature map from the encoder with soft-attention, which has a hidden layer of size 256. The embedding dimension of structural tokens and cell tokens is 16 and 80, respectively. At inference time, the output of both of the decoders are sampled with beam search (beam=3).

The EDD model is trained with the Adam [40] optimizer with two stages. First, we pre-train the encoder and the structure decoder to generate the structural tokens only ($\lambda = 1$), where the batch size is 10, and the learning rate is 0.001 in the first 10 epochs and reduced by 10 for another 3 epochs. Then we train the whole EDD network to generate both structural and cell tokens ($\lambda = 0.5$), with a batch size 8 and a learning rate 0.001 for 10 epochs and 0.0001 for another 2 epochs. Total training time is about 16 days on two V100 GPUs.

B. Quantitative analysis

Table II compares the test performance of the proposed EDD model and the baselines, where the average TEDS of simple[13] and complex[14] test tables is also shown. By solely relying on table images, EDD substantially outperforms all the baselines on recognizing simple and complex tables, even the ones that directly use text extracted from PDF to fill table cells. Camelot is the best off-the-shelf tool in this comparison. Furthermore, the performance of Adobe Acrobat® Pro on image input is dramatically lower than that on PDF input, demonstrating the difficulty of recognizing tables solely on table images. When trained on the PubTabNet dataset, WYGIWYS also considerably outperform the off-the-shelf tools, but is outperformed by EDD by 9.7% absolute TEDS score. The advantage of EDD to WYGIWYS is more profound on complex tables (9.9% absolute TEDS) than simple tables (9.5% absolute TEDS). This proves the great advantage of jointly training two separate decoders to solve structure recognition and cell content recognition tasks.

Input	Method	Average TEDS (%)		
		Simple[13]	Complex[14]	All
PDF	Tabula	78.0	57.8	67.9
	Traprange	60.8	49.9	55.4
	Camelot	80.0	66.0	73.0
	PDFPlumber	44.9	35.9	40.4
	Acrobat® Pro	68.9	61.8	65.3
Image	Acrobat® Pro	53.8	53.5	53.7
	WYGIWYS	81.7	75.5	78.6
	EDD	**91.2**	**85.4**	**88.3**

TABLE II: Test performance of EDD and 7 baseline approaches. Our EDD model, by solely relying on table images, substantially outperforms all the baselines.

C. Qualitative analysis

To illustrate the differences in the behavior of the compared methods, Fig. 6 shows the rendering of the predicted HTML given an example input table. The table has 7 columns, 3 header rows, and 4 body rows. The table header has a complex structure, which consists of 4 multi-row (span=3) cells, 2 multi-column (span=3) cells, and three normal cells.

[9] v0.7.3 (https://github.com/camelot-dev/camelot)

[10] v0.6.0-alpha (https://github.com/jsvine/pdfplumber)

[11] v2019.012.20040

[12] WYGIWYS is trained on the same samples as EDD by truncated back-propagation through time (200 steps). WYGIWYS and EDD use the same CNN in the encoder to rule out the possibility that the performance gain of EDD is due to difference in CNN.

[13] Tables without multi-column or multi-row cells.

[14] Tables with multi-column or multi-row cells.